SKILL - 10

Roll No. : 2520030106

Name : D Mohit Venkat Sai

SEC-8A

Problem Statement :

1. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

2. To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

Aim :

To implement current-directory retrieval, state preservation, exit handling, and environment variable export functionality in C, including validation and propagation of exported variables to a child process.

Procedure :

1. Retrieve the current working directory using **getcwd()** and display the resulting path.

2. Implement an exit routine that records the current directory as saved state, performs cleanup, and terminates the process.

3. Parse an **export NAME=value** command by locating the assignment operator and separating the variable name from its value.

4. Validate the variable name so that it begins with an alphabetic character or underscore and contains only alphanumeric characters or underscores.

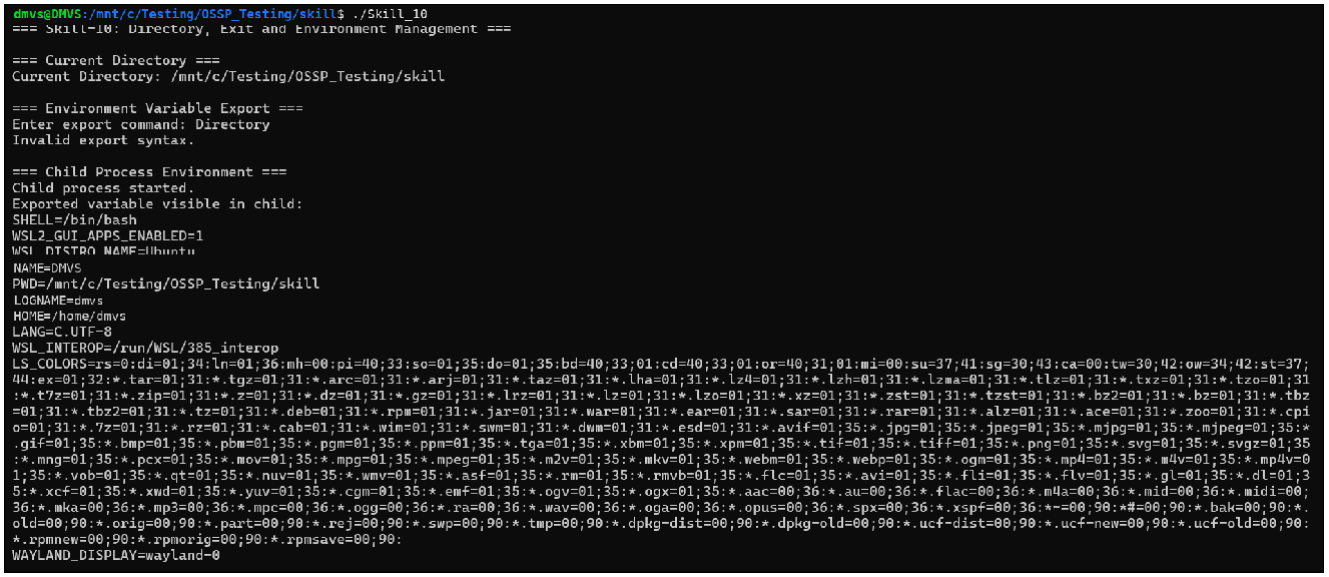
5. Check whether the environment variable already exists and update it using **setenv()**.

6. Create a child process using **fork()** and execute the **env** command in the child using **execlp()**.

7. Use **waitpid()** in the parent to wait for the child process and verify completion.

8. Test the exit request and verify that the saved state and cleanup messages are produced before termination.

Output :



The supplied execution successfully retrieved and displayed the current directory as **/mnt/c/Testing/OSSP_Testing/skill**. The export-input test shown in the terminal used **Directory**, which was rejected with **Invalid export syntax.**.

The child process was then created and executed **env**, displaying the inherited environment. The terminal output shown contains the environment entries and completes with **Child process completed.**.

Finally, the exit test accepted **exit**, displayed the saved working-directory state, performed the cleanup step, and terminated the program.

Source Code - Skill_10.c (1/6):

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <ctype.h>
#include <sys/wait.h>

void show_current_directory()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current Directory: %s\n", cwd);
    else
        perror("getcwd");
}

void process_exit()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("State saved: %s\n", cwd);

    printf("Cleaning up resources...\n");
    printf("Exiting shell...\n");
    exit(0);
}

int valid_variable_name(char *name)
{
    if (name == NULL || name[0] == '\0')
        return 0;

    if (!(isalpha((unsigned char)name[0]) || name[0] == '_'))
        return 0;
```

Source Code - Skill_10.c (2/6):

```c

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!(isalnum((unsigned char)name[i]) || name[i] == '_'))
            return 0;
    }

    return 1;
}

void export_variable(char *input)
{
    char *equal;
    char *name;
    char *value;

    if (strncmp(input, "export", 6) != 0 ||
        (input[6] != ' ' && input[6] != '\t' && input[6] != '\0'))
    {
        printf("Invalid export syntax.\n");
        return;
    }

    char *assignment = input + 6;

    while (*assignment == ' ' || *assignment == '\t')
        assignment++;

    equal = strchr(assignment, '=');

    if (equal == NULL)
    {
        printf("Invalid export syntax. Use export NAME=value\n");
        return;
    }
```

Source Code - Skill_10.c (3/6):

```c

    *equal = '\0';

    name = assignment;
    value = equal + 1;

    if (!valid_variable_name(name))
    {
        printf("Invalid variable name: %s\n", name);
        return;
    }

    if (getenv(name) != NULL)
        printf("Existing variable detected: %s\n", name);

    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        return;
    }

    printf("Environment variable updated successfully.\n");
    printf("%s=%s\n", name, getenv(name));
}

void test_child_environment()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
```

Source Code - Skill_10.c (4/6):

```c
    }

    if (pid == 0)
    {
        printf("Child process started.\n");
        printf("Exported variable visible in child:\n");
        execlp("env", "env", (char *)NULL);
        perror("exec");
        exit(1);
    }

    waitpid(pid, &status, 0);
    printf("Child process completed.\n");
}

int main()
{
    char input[256];

    printf("=== Skill-10: Directory, Exit and Environment Management ===\n\n");

    printf("=== Current Directory ===\n");
    show_current_directory();

    printf("\n=== Environment Variable Export ===\n");
    printf("Enter export command: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    export_variable(input);

    printf("\n=== Child Process Environment ===\n");
    test_child_environment();

    printf("\n=== Exit and State Management ===\n");
    printf("Enter 'exit' to terminate: ");
```

Source Code - Skill_10.c (5/6):

```
145     fgets(input, sizeof(input), stdin);
146     input[strcspn(input, "\n")] = '\0';
147
148     if (strcmp(input, "exit") == 0)
149         process_exit();
150
151     printf("Invalid exit request.\n");
152
153     return 0;
154 }
```

Source Code - Skill_10.c (6/6):

```
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=vinay
DISPLAY=:0
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Eclipse Adoptium/jdk-17.
0.20.101-hotspot/bin:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsPowerShell/v1.0/:/mnt/c/windows/Syst
em32/OpenSSH/:/mnt/c/Program Files/dotnet/:/mnt/c/Program Files/nodejs/:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/Java/jdk-26.0.2.1/bin:/mnt/c/Progr
am Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Users/venka/AppData/Local/Programs/Python/Python311/Scripts:/mnt/c/Users/venka/AppData/Local/Programs/Python/Pyth
on311:/mnt/c/Users/venka/AppData/Local/Programs/Python/Python311/Scripts/:/mnt/c/Users/venka/AppData/Local/Programs/Python/Python311/:/mnt/c/Users/venka/App
Data/Local/Programs/Python/Launcher/:/mnt/c/Users/venka/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/venka/AppData/Local/Programs/Antigravity IDE/bin:/m
nt/c/Users/venka/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/venka/AppData/Roaming/npm:/mnt/c/Users/venka/.lmstudio/bin:/mnt/c/Users/venka/App
Data/Local/Programs/Ollama:/snap/bin
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
HOSTTYPE=x86_64
PULSE_SERVER=unix:/mnt/wslg/PulseServer
OLDPWD=/mnt/c/Testing/OSSP_Testing
_=./Skill_10
Child process completed.

=== Exit and State Management ===
Enter 'exit' to terminate: exit
State saved: /mnt/c/Testing/OSSP_Testing/skill
Cleaning up resources...
Exiting shell...
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$
```

Observation and Analysis

Current Directory and Exit Management

The program uses **getcwd()** to retrieve the current working directory and prints the path for verification. Before termination, the exit routine retrieves the same state, reports it as saved, prints a cleanup message, and then calls **exit()**.

Environment Variable Processing

The export routine accepts the intended **export NAME=value** syntax, locates the '=' separator, separates the name and value, validates the name, checks for an existing environment variable, and updates the environment using **setenv()**.

Child Process Environment

The program creates a child using **fork()**. The child executes **env** through **execlp()**, allowing the environment inherited from the parent to be displayed. **waitpid()** is then used by the parent to wait for child completion.

Observed Test Behavior

The supplied terminal run demonstrates successful directory retrieval, rejection of an invalid export command, creation and completion of the child process, display of the child environment, and controlled program termination with saved state and cleanup messages.

Result :

Skill-10 was successfully implemented in C. The program demonstrates current-directory retrieval, state saving, exit handling, cleanup, export-command parsing, environment-variable validation and updating, and propagation of the process environment to a child process.